

DEEP DIVE: REACTIVE FORMS

REFRESHER

- Formulare bilden oft den Kern von Applikationen
- Zudem besteht häufig die Anforderung nach Validierung und gutem User-Feedback
- Angular liefert volle Unterstützung für alle grundlegenden Anwendungsfälle im Zusammenhang mit Formularen, einschließlich Erweiterungsmöglichkeit (z.B. eigener Validator)
- Angular bietet zwei konzeptionelle Herangehensweisen für Formulare:
 - Model-Driven Forms / ReactiveForms
 - Template-Driven Forms

FORMGROUP, FORMCONTROLS

- Eine Formulargruppe (hier für das gesamte Formular) wird mit ihren Kind-Controls direkt initialisiert.
- Die Controls können dann innerhalb der Gruppe mit ihrem Namen referenziert werden
- Beispiel:

```
export class ProductMdFormComponent {
  readonly productFormGroup = new FormGroup({
    name: new FormControl<string>(''),
    price: new FormControl<number>(0),
    weight: new FormControl<number>(0)
  });
}
```

- Im Template muss das Formular nur noch an das UI gebunden werden
- Die formGroup-Direktive bindet die FormGroup an ein DOMELEMENT
- Die formControlName-Direktive bindet das einzelne Control an ein Input-Element
- Beispiel:

```
<form [formGroup]="productFormGroup">
  <input type="string" formControlName="name">
  <input type="number" formControlName="price">
  <input type="number" formControlName="weight">
  <button (click)="save()">Save</button>
</form>
```

FORMBUILDER

- Formulargruppen können alternativ mithilfe der group-Funktion des FormBuilder Service erstellt werden
- Dabei wird ein Objekt übergeben das die Struktur des Formulars beschreibt
- Der FormBuilder erstellt daraus die FormGroup samt FormControls und gibt diese zurück
- Beispiel:

```
export class ProductMdFormComponent {
  readonly fb = inject(FormBuilder);
  readonly productFormGroup = this.fb.group({
    name: [''],
    price: [0],
    weight: [0]
  });
}
```

VALIDATOREN / CUSTOM VALIDATOREN

- Eigene Validatoren anzulegen ist einfach:

```
export const notZero = (): ValidatorFn => {  
  return (control: AbstractControl) => {  
    if (control.value === 0) {  
      return {  
        notZero: true  
      }  
    }  
    return null;  
  };  
}
```

- Nutzung von Validatoren:

```
export class ProductMdFormComponent {  
  readonly fb = inject(FormBuilder);  
  readonly productFormGroup = this.fb.group({  
    price: [0, [Validators.max(9999), notZero()]],  
  });  
}
```

```
@if (formGroup.controls.price.hasError('notZero')) {  
  <span>  
    Den Preis darf nicht 0 sein  
  </span>  
}  
@if (formGroup.controls.price.hasError('max')) {  
  <span>  
    Den Preis darf 9999 nicht überschreiten  
  </span>  
}
```

DEEP DIVE

MULTI-FIELD-VALIDATOREN

- Mithilfe von Multi-Field-Validatoren können wir mehrere Felder gleichzeitig in eine Validierung mit einbeziehen
- Der Validator wird dafür auf Ebene der FormGroup gesetzt, welche die zu validierenden Felder enthält
 - Dafür bietet die group-Funktion einen zweiten Parameter vom Typ AbstractControlOptions

```
this.empfehlungsForm = fb.group({  
  nameEmpfehlen: [''],  
  nameEmpfohlener: ['', [Validators.required]],  
},  
{validators: CustomValidators.eigenlob}  
);
```

© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

127

- Der Validator bekommt die FormGroup als AbstractControl in die Validator Funktion übergeben
- Aus der FormGroup können nun beliebige Felder und ihre Values gelesen und für die Validierung verwendet werden

```
export class CustomValidators {  
  static eigenlob(ctrl: AbstractControl): ValidationErrors | null {  
    const formGroup = ctrl as FormGroup;  
    const nameEmpfehlen = formGroup.value.nameEmpfehlen;  
    const nameEmpfohlener = formGroup.value.nameEmpfohlener;  
    return nameEmpfehlen === nameEmpfohlener ? {eigenlob: true} : null;  
  }  
}
```

© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

128

- Der Validator schlägt an der FormGroup an und kann über diese abgefragt werden

```
@if(empfehlungsForm.hasError('eigenlob')) {  
  <span>  
    {{ translate('validation.eigenlob') }}  
  </span>  
}
```

© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

129

- Im Validator kann der Fehler aber auch an ein oder mehrere FormControl's gehängt werden

```
export class CustomValidators {
  static eigenlob(ctrl: AbstractControl): null {
    const formGroup = ctrl as FormGroup;
    const {nameEmpfehlen, nameEmpfohlener} = formGroup.controls;

    if (
      nameEmpfehlen &&
      nameEmpfohlener &&
      nameEmpfehlen.value === nameEmpfohlener.value
    ) {
      nameEmpfehlen.setErrors({eigenlob: true});
    } else if (nameEmpfehlen.hasError('eigenlob')) {
      nameEmpfehlen.setErrors({eigenlob: null});
      nameEmpfehlen.updateValueAndValidity();
    }
    return null;
  }
}
```

- Der Validator schlägt nun am FormControl an und kann darüber abgefragt werden

```
@if(empfehlungsForm.controls.nameEmpfehlen.hasError('eigenlob')) {
  <span>
    {{ translate('validation.eigenlob') }}
  </span>
}
```

ASync-VALIDATOREN

- Es können asynchrone Validatoren verwendet werden, um z.B. das Backend für die Validierung mit einzubeziehen
- Dazu muss der Validator eine AsyncValidatorFn zurückliefern
- Der Service der den asynchronen Aufruf macht muss dafür übergeben werden

```
export class BereitsGelobtValidator {
  static createValidator(lobHttpService: LobHttpService): AsyncValidatorFn {
    return (control: AbstractControl): Observable<ValidationErrors | null> => {
      return lobHttpService.isBereitsGelobt(control.value)
        .pipe(
          map((result: boolean) => (result ? {bereitsGelobt: true} : null))
        );
    };
  }
}
```

- Asynchrone Validatoren können als weiteres Argument nach den normalen (synchronen) Validatoren übergeben werden

```
this.empfehlungsForm = fb.group({
  nameEmpfahler: [''],
  nameEmpfohlener: [''],
  [Validators.required],
  [BereitsGelobtValidator.createValidator(lobHttpService)],
});
```

VERSCHACHTELTE FORMULARE

- Es können mehrere FormGroup's ineinander verschachtelt werden.
- Dafür kann bei der Felddefinition auf eine FormGroup verwiesen werden

```
this.empfehlungsForm = fb.group({
  nameEmpfahler: [''],
  adresse: fb.group({
    strasse: [''],
    plz: [''],
  }),
});
```

- Im Template muss der Bereich der geschachtelten FormGroup mithilfe der formGroupName Direktive gekennzeichnet werden

```
<form [formGroup]="empfehlungsForm">
  <input matInput formControlName="confirmPassword"/>
  <div class="new-section" formGroupName="adresse">
    <input formControlName="strasse"/>
    <input formControlName="plz"/>
  </div>
</form>
```

FORMARRAYS

- Um eine mehrfach Eingabe von Daten zu ermöglichen (z.B. die mehrere Adressen in einem Anmeldeformular) können FormArrays benutzt werden
- FormArrays können mithilfe der array-Methode erstellt werden
- Dafür kann bei der Felddefinition auf eine FormGroup innerhalb eines FormArrays verwiesen

```
this.empfehlungsForm = fb.group({
  nameEmpfohlener: [''],
  adressen: fb.array([
    fb.group({
      strasse: [''],
      plz: [''],
    })
  ]),
});
```

- Der Bereich mit dem FormArray kann mithilfe der formArrayName Direktive gekennzeichnet werden
- Mit @for wird über das FormArray iteriert um das Formular für jeden Eintrag im FormArray anzuzeigen
- Mithilfe des index von @for und der FormGroupName Direktive wird noch die jeweilige FormGroup verknüpft

```
@for(adresse of empfehlungsForm.controls.adressen.controls; track adresse; let i = $index;) {
  <div formArrayName="adressen">
    <ng-container [formGroupName]="i">
      <input formControlName="strasse"/>
      <input formControlName="plz"/>
    </ng-container>
  </div>
}
```

- Durch die push-Funktion des FormArrays können weitere Einträge hinzugefügt werden

```
class AddressFormComponent {
  // empfehlungsForm = ...

  addAdresse() {
    const addressFormArray = this.empfehlungsForm.controls.adressen;
    addressFormArray.push(
      this.fb.group({
        strasse: [''],
        plz: [''],
      })
    );
  }
}
```

CONTROLCONTAINER

- Ein in eine Komponente ausgelagertes (Teil-)Formular kann durch den ControlContainer Zugriff auf die umliegende Form erhalten
- Der ControlContainer wird über die Dependency-Injection bezogen

```
export class AddressFormComponent implements OnInit {  
  readonly controlContainer = inject(ControlContainer)  
  
  addressForm?: FormGroup;  
  
  ngOnInit(): void {  
    this.addressForm = this.controlContainer.control as FormGroup;  
  }  
}
```

```
@if(addressForm) {  
  <form [formGroup]="addressForm">  
    <input formControlName="strasse"/>  
    <input formControlName="plz"/>  
  </form>  
}
```

- Die Komponente muss nun noch an entsprechender Stelle ins Formular eingefügt werden

```
<form [formGroup]="empfehlungsForm">  
  <input formControlName="name"/>  
  <div class="new-section" formGroupName="adresse">  
    <address-form/>  
  </div>  
</form>
```